

MOBILE APPLICATION AND DEVELOPMENT

SQLite Databases & Room Persistence Library

SQLite fundamentals · SQLiteOpenHelper and full CRUD · Room as a modern abstraction layer

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science

TODAY'S SESSION

Agenda

01 Why apps need to persist data

02 SQLite — tables, Cursor, SQLiteOpenHelper, and full CRUD

03 Room — Entity, DAO, Database, and using it from an Activity

04 Recap and what's next — Shared Preferences, in Theory 2

INTRODUCTION

Why Apps Need to Persist Data

Every variable in your app lives in RAM — and RAM is wiped the moment the Activity is destroyed, the app is killed, or the phone restarts. Persistence means writing data somewhere that survives all three.

Rotation / navigation

The Activity is recreated — its fields reset unless saved

Process death

Android kills a backgrounded app to free memory

App relaunch

The user closes and reopens the app days later

Login state, draft text, downloaded content, user records — all of it needs a home outside RAM.

PART 1 – SQLITE

What Is SQLite?

An open-source database built into Android that stores structured data as a text file on the user's device. It's offline and local — no connection needs to be opened or managed.

Structured

Data lives in tables, with typed columns

Queryable

Full SQL — filter, sort, join, aggregate

Local & offline

No server, no network — always available

LIKE A SPREADSHEET

Data as Tables

Our example throughout today: a "contacts" table, storing a name and phone number per row, with an auto-incrementing id as the primary key.

id	name	phone_no
1	Habib	568031201
2	Faruk	568031202
3	Mohammad	568031203

WALKING THROUGH RESULTS

Important Cursor Methods

Method	Description
getColumnNames()	Returns the array of column names
getCount()	Returns the number of rows in the cursor
isClosed()	True when the cursor is closed
getColumnCount()	Total number of columns in the table
getColumnIndex(name)	Returns a column's index from its name
getPosition()	The cursor's current row position

SETTING UP THE DATABASE

SQLiteOpenHelper

A class you extend once per database. It manages creation and version upgrades for you — you just fill in two lifecycle methods.

`onCreate(db)`

Runs once, the first time the database is opened — build your tables here

`onUpgrade(db, old, new)`

Runs when DATABASE_VERSION increases — migrate or rebuild your schema

Steps: create the helper class → implement onCreate/onUpgrade → add a method per operation → call it from your Activity.

RULES ON THE DATA ITSELF

Keys and Constraints

A table's **CREATE** statement can enforce rules so bad data never gets in, instead of catching it later in Java.

PRIMARY KEY

One column that uniquely identifies each row — usually an auto-incrementing id

NOT NULL

This column must always have a value — an insert without it fails

UNIQUE

No two rows can share this value — e.g. one phone number per contact

```
CREATE TABLE contacts (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, phone_no TEXT UNIQUE);
```

WHEN THE SCHEMA CHANGES

Versioning with onUpgrade()

SQLiteOpenHelper's constructor takes a version number. Bump it, and Android calls onUpgrade() the next time the app runs — your one chance to migrate old data before the new schema takes over.

```
public class MyDBHelper extends SQLiteOpenHelper {  
    private static final int VERSION = 2;  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {  
        db.execSQL("ALTER TABLE contacts ADD COLUMN email TEXT");  
    }  
}
```

Forget to bump the version and your ALTER never runs — a very common source of "it works on my emulator" bugs.

CREATE THE TABLE, THEN INSERT

MyDBHelper.java

```
public class MyDBHelper extends SQLiteOpenHelper {
    private static final String TABLE_CONTACT = "contacts";
    public MyDBHelper(Context c) { super(c, "ContactDB", null, 1); }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE_CONTACT + "(" +
            "id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, phone_no TEXT)");
    }

    public void addContact(String name, String phone) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put("name", name);
        values.put("phone_no", phone);
        db.insert(TABLE_CONTACT, null, values);
    }
}
```

TURNING ROWS BACK INTO OBJECTS

Reading Data with a Cursor

```
public ArrayList<ContactModel> fetchContact() {  
    SQLiteDatabase db = this.getReadableDatabase();  
    Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_CONTACT, null);  
    ArrayList<ContactModel> list = new ArrayList<>();  
    while (cursor.moveToNext()) {  
        ContactModel m = new ContactModel();  
        m.id = cursor.getInt(0);  
        m.name = cursor.getString(1);  
        m.contact = cursor.getString(2);  
        list.add(m);  
    }  
    return list;  
}
```

`getReadableDatabase()` for reads, `getWritableDatabase()` for writes — `moveToNext()` advances the cursor one row at a time.

CHANGING AN EXISTING ROW

Updating a Contact

```
public void updateContact(ContactModel model) {  
    SQLiteDatabase db = this.getWritableDatabase();  
    ContentValues cv = new ContentValues();  
    cv.put("phone_no", model.contact);  
    db.update(TABLE_CONTACT, cv, "id = " + model.id, null);  
}
```

The third argument is the SQL WHERE clause — it decides which row(s) get the new values. Leave it null (with no id filter) and you'd update every row.

REMOVING A ROW

Deleting a Contact

```
public void deleteContact(int id) {  
    SQLiteDatabase db = this.getWritableDatabase();  
    db.delete(TABLE_CONTACT, "id = ?", new String[]{String.valueOf(id)});  
    db.close();  
}
```

Notice the "?" placeholder instead of gluing id into the string directly — that's a parameterized query, and it's what keeps user input from being interpreted as SQL.

THE FOUR OPERATIONS

SQLite CRUD, in One Table

Operation	SQLiteDatabase call	Our method
Create	db.insert(table, null, values)	addContact()
Read	db.rawQuery(sql, args)	fetchContact()
Update	db.update(table, values, where, args)	updateContact()
Delete	db.delete(table, where, args)	deleteContact() — your turn to write it

ALL OR NOTHING

Transactions

Inserting 100 rows one at a time is slow — each is its own disk write. Wrapping them in a transaction batches the writes, and if anything fails partway, none of it is applied.

```
db.beginTransaction();
try {
    for (ContactModel c : contacts) {
        db.insert(TABLE_CONTACT, null, toValues(c));
    }
    db.setTransactionSuccessful();
} finally {
    db.endTransaction();
}
```

`setTransactionSuccessful()` must be the last call before `endTransaction()` — skip it and everything rolls back.

LEARN FROM THESE BEFORE YOU HIT THEM

Common SQLite Mistakes

Not closing the Cursor

Every query() returns a Cursor holding a database connection open — call cursor.close() when done

Database work on the main thread

A slow query freezes the UI and can trigger an ANR — move it to a background thread

Building SQL with string concatenation

"WHERE id=" + userInput opens the door to SQL injection — always use "?" placeholders

Forgetting to bump the DB version

onUpgrade() never fires, so a changed schema silently doesn't apply on existing installs

PART 2 – ROOM

A Modern Layer over SQLite

Room is a persistence library that sits on top of SQLite — it writes the SQLiteOpenHelper boilerplate you just saw for you, checks your queries at compile time, and gives you three clear building blocks.

Entity

A class annotated `@Entity` — one table, one row per object

DAO

An interface with your queries — Room writes the implementation

Database

An abstract class that ties the entities and DAOs together

```
implementation("androidx.room:room-runtime:2.6.1")  
annotationProcessor("androidx.room:room-compiler:2.6.1")
```

STEP 1 – THE TABLE, AS A CLASS

The Entity

```
@Entity(tableName = "expenses")
public class Expense {
    @PrimaryKey(autoGenerate = true)
    public int id;

    @ColumnInfo(name = "title")
    public String title;

    @ColumnInfo(name = "amount")
    public double amount;

    Expense(String title, double amount) {
        this.title = title;
        this.amount = amount;
    }
}
```

STEP 2 – THE QUERIES, AS AN INTERFACE

The DAO

```
@Dao
public interface ExpenseDao {
    @Query("select * from expenses")
    List<Expense> getAllExpenses();

    @Insert
    void insertExpense(Expense expense);

    @Update
    void updateExpense(Expense expense);

    @Delete
    void deleteExpense(Expense expense);
}
```

You write the signature and the SQL. Room generates the implementation at compile time — no manual Cursor loop needed.

STEP 2B – QUERIES WITH PARAMETERS

Passing Values into @Query

```
@Query("select * from expenses where amount > :minAmount")  
List<Expense> getExpensesAbove(double minAmount);
```

```
@Query("select * from expenses where title like :search")  
List<Expense> searchExpenses(String search);
```

A ":name" in the SQL string binds straight to the method parameter of the same name — Room checks this matches at compile time, so a typo is caught before you ever run the app.

STEP 2C – STORING NON-PRIMITIVE FIELDS

TypeConverters

SQLite only knows a handful of column types. To store a Date, a List, or any other object on an @Entity field, tell Room how to convert it to and from something it can save.

```
public class Converters {  
    @TypeConverter  
    public static Date fromTimestamp(Long value) {  
        return value == null ? null : new Date(value);  
    }  
    @TypeConverter  
    public static Long toTimestamp(Date date) {  
        return date == null ? null : date.getTime();  
    }  
}
```

Register it on the database class: @TypeConverters(Converters.class)

STEP 3 – TYING IT TOGETHER

The Database

```
@Database(entities = Expense.class, version = 1, exportSchema = false)
public abstract class DatabaseHelper extends RoomDatabase {
    private static DatabaseHelper instance;

    public static synchronized DatabaseHelper getDb(Context context) {
        if (instance == null) {
            instance = Room.databaseBuilder(context,
                DatabaseHelper.class, "expensedb").build();
        }
        return instance;
    }

    public abstract ExpenseDao expenseDao();
}
```

STEP 4 – USING IT FROM AN ACTIVITY

Insert and Query

```
DatabaseHelper db = DatabaseHelper.getDb(this);

db.expenseDao().insertExpense(
    new Expense(title, amount)
);

List<Expense> all = db.expenseDao().getAllExpenses();
```

Compare this to [sqlite-read-code](#) — no `rawQuery()`, no `Cursor`, no manual column mapping. Room does what `MyDBHelper` did by hand.

STEP 5 – THE UI UPDATES ITSELF

Observing Data with LiveData

Return LiveData from a DAO query instead of a plain List, and Room notifies your observer automatically whenever the underlying table changes — no manual re-query after every insert.

```
@Query("select * from expenses")
LiveData<List<Expense>> getAllExpensesLive();

// in the Activity:
db.expenseDao().getAllExpensesLive().observe(this, list → {
    adapter.setData(list);
});
```

STEP 6 – CHANGING THE SCHEMA SAFELY

Migrations

Bump the version in `@Database` and, without a migration, Room deletes and recreates every table – losing all existing data. A Migration describes the raw SQL to run instead.

```
static final Migration MIGRATION_1_2 = new Migration(1, 2) {  
    @Override  
    public void migrate(SupportSQLiteDatabase db) {  
        db.execSQL("ALTER TABLE expenses ADD COLUMN note TEXT");  
    }  
};
```

Register it with `Room.databaseBuilder(...).addMigrations(MIGRATION_1_2)`. In a class project, `.fallbackToDestructiveMigration()` is an acceptable shortcut – never in a real app with user data.

PART 4 – RECAP

SQLite vs. Room, Side by Side

Writing queries	SQLite: raw SQL strings and <code>rawQuery()</code> · Room: method calls on a <code>@Dao</code> interface
Reading results	SQLite: manual <code>Cursor</code> + column mapping · Room: typed objects returned directly
Compile-time safety	SQLite: none — typos surface at runtime · Room: SQL is verified when you build
When to use which	Learn SQLite first to understand what's underneath — build new apps with Room

BEFORE NEXT CLASS

What to Do This Week

Build the contacts demo

Rebuild MyDBHelper from today, add your own deleteContact()

Try the Room example

Rebuild the Expense entity/DAO/database and insert a few rows

Compare both approaches

Write the same query both ways — feel the difference Room makes

Next lecture (Theory 2): Data Storage & Persistence — Shared Preferences for small key/value settings.

Questions?

SQLite · SQLiteOpenHelper · CRUD · Room (Entity, DAO, Database)

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science